Exercises

Exercise 7: Reproducible reports with R markdown

Read Chapter 8 to help you complete the questions in this exercise.

Before the session. This exercise needs the `rmarkdown` and `knitr` packages, a LaTeX installation so that you can make PDF documents, and `ggplot2`, which you installed for Exercise 6. Work through **Setting up R markdown**, under the Setup menu above, before you come to the session. It takes about fifteen minutes and most of that is a download running by itself, but please don't leave it until the morning of the session. If you hit a real problem we'll help you at the start of the session, but we can't get everybody set up and still have time to teach anything.

Up to now everything you've written has been code, for your own use. This exercise is about producing a *document* for somebody else to read, in which your writing, your code and your results all live in the same R markdown file. Because they're in the same file they can't drift apart, which is what we mean by a reproducible report. Change the data, press one button, and every number, figure and table in the output file updates itself.

It's also the exercise where the rest of the course comes together. You won't be learning much new R here. Instead you'll be taking the things you already know how to do, importing data, checking it for values that can't be right, transforming a variable, drawing a plot and summarising into a table, and setting them out in the order a reader needs them: what the data are, what you did to them and why, and what you found. That order is the point. A report is not a pile of results, it's an account of a piece of work.

The figures in this exercise use `ggplot2`, the way Exercise 6 did. If you'd rather draw them with base R then please do, and nothing else here changes. That's one of the nice things about R. Nobody makes you work in one particular way, so use whichever you're quickest and most confident with.

You'll build **one document** across the whole exercise, adding something to it at each question, so don't start a new file each time. There's a lot of new vocabulary here, so take it slowly and knit often. Knitting after every small change is the quickest way to see what each thing does, and the quickest way to find out which change broke something.

1. Open the RStudio Project you have been using for this course. Create a new R markdown document by going to `File -> New File -> R Markdown...` (see Section 8.4 of the Introduction to R book if you would like to read about this first).

The first time you do this, RStudio may tell you it needs to install some packages. Let it, and wait for it to finish.

In the dialogue box that appears, type `Cardiac study report` in the Title box, put your name in the Author box, choose **PDF** as the default output format, and click OK. RStudio opens a new document that's already full of example content about cars. That's normal, and we'll delete it in a moment.

Before you change anything at all, save the file into your Project directory as `cardiac_report.Rmd` (`File -> Save As...`), then click the **Knit** button at the top of the editor. You can also press Ctrl+Shift+K, or Cmd+Shift+K on a Mac.

Do this before you understand any of it. If a PDF appears then your setup is working, and you can be confident that anything which goes wrong later is something you can fix.

2. Look at the PDF that appeared alongside the R markdown file that produced it. An R markdown file has only three kinds of content, and everything else is a variation on them (Section 8.5 of the book takes each of them in turn):

- the **YAML header** at the very top, between the two lines of `---`, which controls the document as a whole
- **text**, written in a simple markup language called markdown
- **code chunks**, which look like this:

````
```{r}
mean(cardiac$age)
```
````

Find one of each in `cardiac_report.Rmd`, the file you are editing rather than the PDF it produced, so that you can recognise them when we start changing them. Then delete everything in `cardiac_report.Rmd` below the first code chunk, the one called `setup`, leaving the YAML header and that chunk in place. Knit again to check you have an empty but working document. This is your starting point.

3. Now take control of the YAML header (see Section 8.5.1 of the book). Find the `output:` line, which currently says `pdf_document` on the same line as it, and turn it into the indented block shown below.

Two warnings, because this is where people lose ten minutes. Indentation in YAML matters, so copy the spacing exactly. And use spaces, never tabs.

Build it up rather than pasting it in one go. Put `pdf_document:` on its own line first, with `toc: true` indented underneath, and knit. Then add `number_sections: true` below that and knit again. Doing it one line at a time, and knitting after each, is how you see what each one actually does. Here is what you are aiming for:

```yaml
---
title: "Cardiac study report"
author: "Your Name"
date: "2 November 2026"
output:
  pdf_document:
    toc: true
    number_sections: true
---
```

4. Every report starts by telling the reader what the data are, so that's where you'll start too. You already have a description of these data: it's the one you read in **Exercise 3, Q4**. Copy the first few sentences of it into your document underneath the setup chunk, then use markdown to format it. There is no code in this question at all.

- put a heading above it, written as `# Introduction`, and a second smaller heading below that, written as `## Data description`
- put **cohort study** in bold, by writing it as `**cohort study**`
- put *measurements* in italics the first time it appears, by writing it as `*measurements*`
- pick four of the variables the description mentions and set them out as a bulleted list at the end, each line starting with a dash and a space

That's enough to be going on with. If you want to add a link, a numbered list, a quotation or a table, Section 8.5.2 of the book has the syntax for all of them, and there's a summary on the R markdown cheat sheet under `Help -> Cheat Sheets`.

Knit, and put the R markdown file and the PDF side by side. Notice that the headings you wrote have appeared in the table of contents on their own, because of the `toc: true` you added in Q3.

5. Next comes the data. Insert a new code chunk (see Section 8.5.3 of the book) by going to `Code -> Insert Chunk`, or with the keyboard shortcut Ctrl+Alt+I, which is Cmd+Alt+I on a Mac. An empty chunk appears. If you'd rather, there's nothing magic about the menu: you can simply type `` ```{r} `` on a line of its own and `` ``` `` on another line below it, and anything you put between them is a chunk.

Give the chunk a name by typing it straight after the `r`, so the first line reads `` ```{r import} ``. Naming chunks costs nothing and makes them far easier to find when something goes wrong.

Inside the chunk, import `cardiacdata.txt` exactly as you did in **Exercise 3, Q5**: with the `read.table()` function, writing out the `header`, `sep` and `stringsAsFactors` arguments, and assigning the result to `cardiac`. In the same chunk, add the two `factor()` lines from **Exercise 3, Q7** that create `Fsex` and `Fsmoking`. You'll remember why. `sex` and `smoking` arrive from the file as the numbers they were coded as, 1 and 2 for sex and 1, 2 and 3 for smoking, but they are really categories rather than quantities, so you use `factor()` to attach a meaningful label to each code and leave the original columns untouched (see `?factor` and Section 3.1 of the book for where factors sit among R's data types). You'll need `Fsmoking` for the boxplot in Q7 and the table in Q8, and both of them read a great deal better with words in them than with numbers.

One more thing while you're in there. Later in this exercise you'll want a couple of functions from the `knitr` package, and you'll want `ggplot2` for the figures, and the tidy place for a `library()` call is at the top of a document rather than buried in the middle of it. Add `library(knitr)` and `library(ggplot2)` to your `setup` chunk now, the one you kept back in Q2, and you won't have to think about them again.

Run the import chunk in the editor with the small green arrow at its top right corner, the rightmost of the three little buttons there. If you can't see it, `Code -> Run Region -> Run Current Chunk` does the same job, and so does Ctrl+Alt+C, or Cmd+Alt+C on a Mac. Then knit the whole document. Notice that knitting starts a completely fresh R session: anything you made earlier in the console doesn't exist when the document knits, which is exactly what makes it reproducible. It also means the chunk that imports the data has to come before any chunk that uses it.

6. Now you have the data imported, so the next thing a reader needs to know is whether you can trust it, and what you did about the bits you couldn't. This question asks you not just to do something to the data but to explain why, which is most of what report writing is.

Add a new heading called `## Data validation` below the previous chunk. Then build the section up in three pieces.

First, one sentence directly under the heading saying how many patients are in the dataset, with the number written in as an ordinary number. You'll come back to that sentence in Q9.

Second, below it, a chunk named `validation` holding the three lines from **Exercise 4, Q1** that set the impossible `bmi`, `triglyceride` and `hdlchol` values to `NA`. Remember that your report imports the raw file from scratch every time it knits, so if the cleaning isn't in the document then it hasn't happened.

Third, beneath that chunk, a short paragraph of two or three sentences saying which variables you changed, how many values in each, and why you set them to `NA` rather than deleting those patients or correcting the numbers yourself. Write the counts in as ordinary numbers too, so something like 'two patients had an HDL cholesterol of 0 mmol/l'.

You'll remember from **Exercise 5, Q9** that the variable `alcohol` needed some work as well. A small number of patients drink a great deal more than the rest, which leaves the distribution with a long tail to the right, and you took the square root to pull that tail in. That's a data preparation step exactly like the cleaning above, so it belongs in this section too. Add a chunk named `alcohol-transform` that creates the square root of `alcohol` and then draws two histograms, one of `alcohol` and one of the square root, so a reader can see what the transformation did. Write a sentence or two underneath saying why you did it.

Two plots in one chunk come out one above the other by default. To sit them side by side, add `fig.show = 'hold'` and `out.width = '50%'` to the chunk options, in the curly braces after the chunk name. That's R markdown laying the page out for you rather than anything to do with the plotting, so it works whichever package you draw with.

7. With the data in and tidied up, you can start showing the reader what's in it. Insert another chunk, name it `chol-plot`, and use it to draw a boxplot of HDL cholesterol by smoking status. It's the same comparison you drew in **Exercise 5, Q11**, built with `ggplot2` this time: `Fsmoking` along the x axis, `hdlchol` up the y axis, and `geom_boxplot()` as the geom. You already have `Fsmoking` from your import chunk.

One thing to watch out for. The seven patients with no recorded smoking status get a box of their own, labelled NA, which isn't something you want in front of a reader. Adding a `scale_x_discrete(na.translate = FALSE)` layer leaves them out.

Now give the figure a caption. Chunk options go inside the curly braces, after the chunk name, separated by commas. The first line of your chunk should read:

````
```{r chol-plot, fig.cap = 'HDL cholesterol by smoking status.'}
```
````

Knit and find your caption underneath the plot. Then add `fig.width = 4` to the same line, knit again, and see what happens to the size of the text relative to the plot.

While you're at it, go back to your `alcohol-transform` chunk from Q6 and give that figure a caption too, so both of your figures have one.

There are a great many other chunk options for figures, controlling height, alignment, resolution and so on. You don't need them today. Section 8.5.4 of the book covers the ones you're most likely to need.

8. A figure visualises your data, but a reader will usually want tabulated summaries as well, so add a table to go alongside it (see Section 8.5.5 of the book).

Insert a chunk, name it `summary-table`, and recreate the `aggregate()` summary from **Exercise 4, Q4**: the mean of `age`, `systolic` and `tchol` for each smoking category. Assign it to an object called `smoke_summary`. Then, on the next line down, write `smoke_summary` on its own. A chunk prints anything that isn't assigned to something, so that second line is what makes the summary appear in your document at all. Knit and see what you get.

It's the console output, monospaced and squeezed into a grey box, which is fine while you're working but not something you would put in front of anybody else. The `kable()` function, from the `knitr` package you loaded in Q5, turns a dataframe into a properly formatted table instead, with the columns lined up and the variable names picked out as a header row. Change that second line from `smoke_summary` to `kable(smoke_summary)`, knit again, and compare the two. Finally add `digits = 1` and a `caption` to the `kable()` call to tidy it up.

9. Your report is now more or less complete, so the rest of the exercise is about tidying it up. Start with the sentence you wrote at the top of your data validation section in Q6, the one saying how many patients are in the dataset. You wrote 163 into it yourself, and a number written into your text that way goes stale. If the data changes, or you correct something later, your writing quietly stops matching your results and nothing tells you.

R markdown fixes this with **inline code** (see Section 8.5.6 of the book). In the middle of an ordinary sentence you write a backtick, then the letter `r`, then some R code, then a closing backtick. When you knit, the answer of that code appears in your text in place of the whole thing:

```
The dataset contains `r nrow(cardiac)` patients.
```

which comes out as "The dataset contains 163 patients."

a) Go back to that sentence and swap the number 163 for `` `r nrow(cardiac)` ``. Knit and check it still says the same thing.

b) Now add the mean age to the same sentence, worked out the same way with `mean()`. Wrap it in `round()` so you don't get six decimal places.

c) Now add a second sentence saying how old the youngest and oldest patients were, using `min()` and `max()` inside inline code. Put `round()` around each one so you don't get the decimals. Knit and compare what you get with the age range given in the description of the data back in **Exercise 3, Q4**.

Every one of these numbers is worked out from `cardiac` each time you knit, so none of them can drift out of step with the data the way a number you typed in would.

10. Now think about who's going to read your report, because it decides how much of your working they should see. This one is a technical document, written for people who know R, so showing the code is exactly right and everything you've done so far is fine as it is. But you'll often write for readers who don't know R at all, a clinical team, a service manager, a patient group, and for them a wall of code is just something to scroll past on the way to the results.

You control this chunk by chunk with the `echo` option, in the same curly braces as the caption you added in Q7. Section 8.5.3 of the book lists the chunk options alongside it. Add `echo = FALSE` to your `chol-plot` chunk and knit. The plot is still there, but the code that drew it is not. Take it off again afterwards, since we want the code visible in this report.

Code isn't the only thing that can end up in your document without you asking for it. **Messages** are the chatty output R produces while it's doing something in the background, most often when you load a package with `library()`, and they can easily run to half a dozen lines. **Warnings** are R telling you that something might not be right, and they tend to look a lot more alarming to a reader than they usually deserve to. You've met one already: when ggplot drew your boxplot it warned that it had removed four rows, because four patients have no HDL cholesterol value. That's worth knowing about and it isn't worth showing a reader. Both messages and warnings land in the middle of your writing. You switch them off in the same way as the code, with `message = FALSE` and `warning = FALSE`, either on a single chunk or on all of them at once by putting them inside the `knitr::opts_chunk$set()` call in your `setup` chunk.

Be careful with `warning = FALSE` though. You're hiding warnings from your reader, not from yourself, so keep an eye on the ones appearing in the console while you work, because some of them matter.

11. Not every figure in a report will be one you constructed with R code. You might need a diagram, a photograph, a map, or a figure somebody else made (see Section 8.5.4 of the book again). Add the plot you exported in **Exercise 6, Q9**, `ex6_admissions.png`, to your report under a heading of its own. If you no

longer have the file, download it here and save it into your `output` directory, the one you created in **Exercise 1, Q12**.

There are two ways of doing it and it's worth trying both.

a) The quick way needs no chunk at all, just a line typed in among your text. It looks like this, and you replace the two parts in capitals with your own:

```
![YOUR CAPTION](PATH/TO/YOUR/FILE.png)
```

An exclamation mark, then square brackets, then round brackets, with no spaces anywhere between them. Knit and check the figure appears.

b) The other way puts the file into a chunk instead, using the `include_graphics()` function, which is another one from the `knitr` package you loaded in Q5. On its own that looks like more work for the same result, but it buys you something. Because the figure is now coming out of a chunk, all the chunk options are available to it, so you can caption it with `fig.cap` exactly as you did in Q7 and set how much of the page width it takes up with `out.width`. Again, replace the parts in capitals:

````
```{r YOUR-CHUNK-NAME, out.width = "50%", fig.cap = 'YOUR CAPTION'}
include_graphics("PATH/TO/YOUR/FILE.png")
```
````

Try it, and have a play with the `out.width` percentage until the figure sits sensibly on the page.

c) Now write two or three sentences underneath it. Say what the figure shows, where it came from, and what a reader should take away from it. A figure dropped into a report with nothing said about it leaves the reader to work out for themselves why it is there, which is your job rather than theirs.

Typing the line straight into your text is quicker, and putting it in a chunk gives you control over how the figure is presented, which is usually what you want in a report.

One thing that catches everybody out: the path is relative to where the `.Rmd` file is, not to your working directory. If the image doesn't appear, the path is almost always the reason.

12. If you have time, here's one of the nicer things about R markdown: the same R markdown file will give you a completely different kind of document without you rewriting a word of it. In your YAML header, change `pdf_document` to `html_document` and knit. You'll get a web page instead, built from exactly the same text, code and figures. While you're there, add `toc_float: true` on its own line underneath `toc: true` and knit again to see the contents float alongside the page as you scroll. That option only works in HTML, so change the header back to `pdf_document` and delete the `toc_float` line before you finish.

You don't have to edit the YAML to do this. Click the small black triangle next to the **Knit** button and you can pick a format straight from the menu, which overrides whatever your YAML header says for that one knit. It's quicker for a look, but the YAML is where you set what your document actually is.

Two things we haven't covered

The visual editor. RStudio can show an R markdown document formatted as you type, more like a word processor, hiding the markdown syntax. Look for the pair of buttons marked **Source** and **Visual** at the top left of the editor toolbar, just above your document, and click **Visual**. The keyboard shortcut is Ctrl+Shift+F4, or Cmd+Shift+F4 on a Mac. It's genuinely useful, especially for tables and links. We've left it until now on purpose, so that you learn what the markdown underneath actually looks like first, because when something goes wrong it's the markdown you have to fix.

Quarto. Quarto is the successor to R markdown. It does the same job, works with languages other than R, and its syntax is so close that nearly everything in this exercise transfers unchanged. If you carry on using R after this course you'll meet it. Start at quarto.org.

End of Exercise 7